Guide de déploiement

Installation, services, HTTPS et mise à jour

Suite de 12 modules : hotel-tryton, hotel_pms-tryton, hotel_housekeeping-tryton, hotel_channel_manager-tryton - version 7.0

1. Prérequis

- Python 3.8 ou plus récent ; PostgreSQL 13 ou plus (16 testé).
- trytond 7.0 et les modules officiels company, currency, party, product, stock, sale, sale_discount, sale_point, sale_subscription, purchase, purchase_request, stock_supply, account, account_invoice (série 7.0).
- Un client Tryton 7.0 (bureau ou web tryton-sao).

2. Installation des paquets

Les paquets s'appellent hotel-tryton, hotel_pms-tryton, hotel_housekeeping-tryton et hotel_channel_manager-tryton. Depuis les sources :

```
python -m venv venv && . venv/bin/activate
pip install trytond==7.0.* trytond-sale trytond-purchase \
    trytond-sale-point trytond-sale-subscription \
    trytond-sale-discount trytond-purchase-request \
    trytond-stock-supply trytond-account-invoice
pip install ./hotel ./hotel_pms ./hotel_housekeeping \
    ./hotel_channel_manager
```

Alternative sans pip : copier les quatre dossiers dans trytond/modules/.

Pour les nouvelles installations, adaptez les versions officielles à la série 7.0 (trytond-sale>=7.0,<7.1, etc.).

3. Configuration du serveur

```
[database]
uri = postgresql://tryton:***@localhost:5432/
path = /var/lib/trytond

[web]
listen = 0.0.0.0:8000

[product]
price_decimal = 2

[bus]
allow_subscribe = True
```

4. Création de la base

```
createdb -O tryton hotel
trytond-admin -c trytond.conf -d hotel \
    -u hotel hotel_pms hotel_housekeeping \
    hotel_channel_manager --activate-dependencies -l fr
trytond-admin -c trytond.conf -d hotel --email admin@exemple.com
```

5. Services

Lancer deux processus : le serveur web et le démon des tâches planifiées (indispensable aux notifications, au ménage automatique et au Channel Manager).

```
trytond -c trytond.conf
trytond-cron -c trytond.conf -d hotel
```

Exemple d'unité systemd (fichier /etc/systemd/system/trytond.service) :

```
[Unit]
Description=Tryton hotel
After=postgresql.service

[Service]
User=tryton
ExecStart=/opt/tryton/venv/bin/trytond -c /etc/trytond.conf
Restart=always

[Install]
WantedBy=multi-user.target
```

Créer une seconde unité identique pour trytond-cron.

6. Reverse proxy et HTTPS

Placer nginx ou Caddy devant trytond avec un certificat TLS. Pour le Channel Manager, transmettre aussi le chemin /<base>/hotel/ical/ au serveur : les sites de réservation lisent le calendrier d'export par HTTPS.

```
location / {
    proxy_pass http://127.0.0.1:8000;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

7. Client web

Déployer le client web Tryton (sao) pour les tablettes et téléphones du personnel d'étage ; les vues compactes du ménage (Mes tâches) y sont prévues.

8. Mise à jour

1. Sauvegarder la base (pg_dump).
2. Arrêter le serveur et le cron.
3. Remplacer les modules ou réinstaller les paquets.
4. Exécuter trytond-admin ... -u <modules> -l fr.
5. Redémarrer et vérifier tools/check_actions.py BASE.

Migration depuis d'anciens paquets nommés trytond_hotel* : désinstaller l'ancien paquet pip (pip uninstall trytond_hotel trytond_hotel_pms ...) avant d'installer les paquets *-tryton, pour éviter deux copies du même module. Les données ne sont pas affectées : le nom du module reste identique.

9. Liste de contrôle de mise en production

- Sauvegardes automatiques testées (restauration comprise).
- HTTPS actif, mot de passe admin changé.
- trytond-cron actif.
- Hôtel, chambres, tarifs et articles saisis.
- Utilisateurs et groupes attribués.
- Test complet : réservation, arrivée, consommation, départ, facture.

- Tests unitaires passés sur l'environnement cible.

10. Installer la suite complète

```
pip install ./hotel ./hotel_pms ./hotel_housekeeping \
    ./hotel_channel_manager ./hotel_rates_plus ./hotel_billing \
    ./hotel_guest ./hotel_groups ./hotel_maintenance \
    ./hotel_services ./hotel_communication ./hotel_booking_engine

trytond-admin -c trytond.conf -d hotel \
    -u hotel hotel_pms hotel_housekeeping hotel_channel_manager \
    hotel_rates_plus hotel_billing hotel_guest hotel_groups \
    hotel_maintenance hotel_services hotel_communication \
    hotel_booking_engine --activate-dependencies -l fr
```

Chaque module est optionnel (hotel_communication demande hotel_guest). Les pages publiques (/<base>/hotel/booking/ et /<base>/hotel/precheckin/) doivent être transmises par le reverse proxy en HTTPS. Le module country est installé comme dépendance de hotel_guest et hotel_communication.

Hôtel : déploiement des liaisons

Mise à jour v3 — ajout au guide d'origine. Auteur : Pierre Michel Augustin — AGPLv3.

```
pip install mobile_payment-tryton moncash-tryton natcash-tryton (installe aussi segno pour
le code QR ; natcashify requiert Python 3.12+)
```

```
pip install hotel_restaurant-tryton hotel_mobile_payment-tryton hotel_sale_point-tryton
```

```
trytond-admin -d <base> -u hotel_restaurant hotel_mobile_payment hotel_sale_point
--activate-dependencies
```

Ordre : party_person_ht, hotel, hotel_pms, hotel_billing, restaurant, mobile_payment, moncash, natcash, puis les modules de liaison. Déclarer chez l'opérateur l'URL de retour `https://<serveur>/<base>/mobile_payment/<opérateur>/return` (HTTPS obligatoire en production). Faire un essai complet en sandbox avant la mise en service.

Thème des clients : voir ui/ (SAO : custom.css ; desktop : theme.css).